# EdgeFlow AI

## Consolidated Revised Development Plan

Current engineering baseline | 8 September 2026

**EdgeFlow AI | Team 31µF | VEDC 2026**

# 1. Project Direction and Architecture

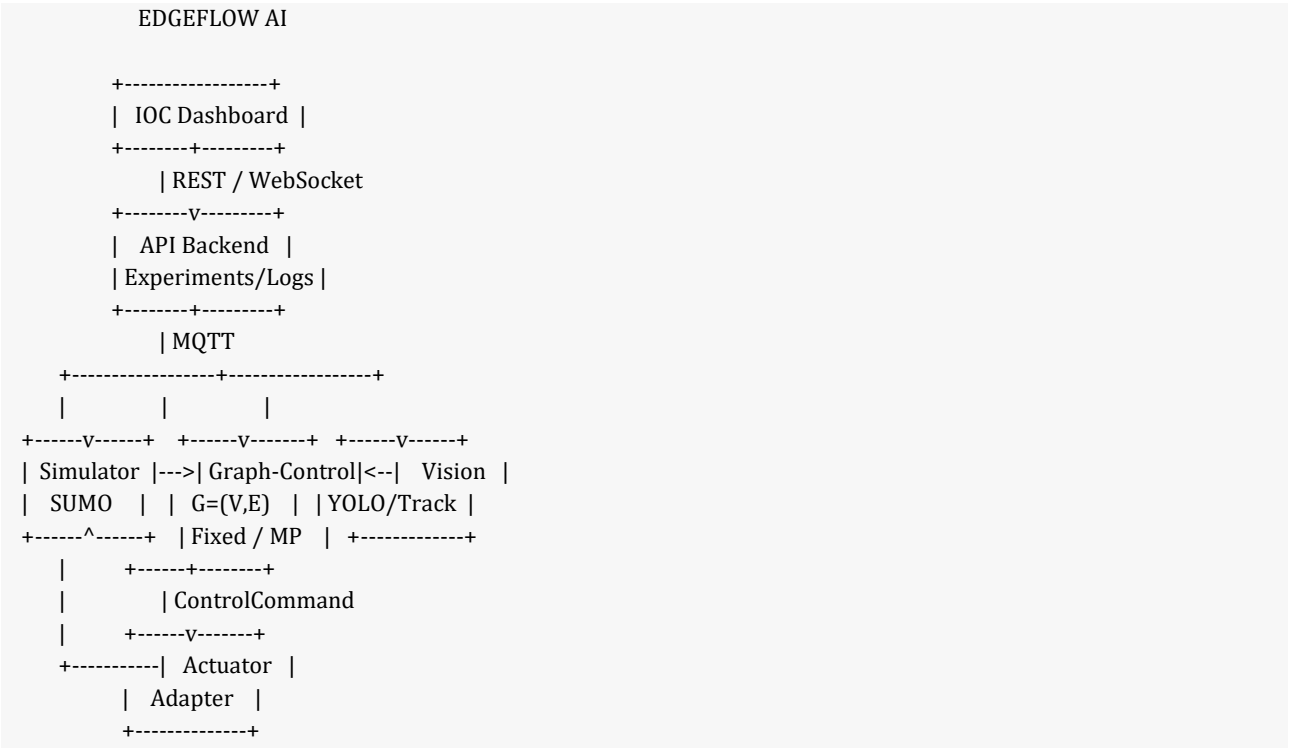
## 1.1 Objective

EdgeFlow AI will demonstrate adaptive traffic-signal control using network-level traffic state instead of fixed timing. The system models a road network as a directed graph G=(V,E), ingests normalized traffic observations, calculates safe signal decisions, and compares adaptive operation quantitatively against a Fixed-Time baseline.

- Represent intersections, approaches and road segments using a canonical directed topology.
- Use queue, flow, speed and occupancy as normalized traffic-state features.
- Use Max-Pressure as the primary adaptive controller; retain RL/PressLight/MARL as optional extensions only after the baseline is stable.
- Support simulated, vision-derived and future physical telemetry through the same contracts.
- Support virtual, SUMO and future physical signal actuation through the same control contract.
- Use Docker Compose microservices for the software stack and reproducible integration testing.

**Architectural invariant: Graph-Control must not know whether telemetry came from SUMO, video, Raspberry Pi, or future physical sensors; actuator implementations must consume the same abstract control command.**

## 1.2 Final logical architecture




## 1.3 Scope changes from the previous plan

| Previous assumption | Current baseline |
|---|---|
| Raspberry Pi + 3 ESP32 sensing nodes | Camera-centric sensing; no ESP32 dependency |
| 12 IR LM393 sensors | Removed from the critical path |
| ESP-NOW inter-node communication | Removed; MQTT is the software event backbone |

| Previous assumption | Current baseline |
|---|---|
| TFLite Micro policy on ESP32 | Removed |
| Distributed/edge-to-edge control | Centralized Graph-Control service |
| Hardware required from Week 1 | Weeks 1-2 are software/digital-twin development |
| RL/MARL central to implementation | Max-Pressure required; RL is stretch scope |
| Ad-hoc JSON interfaces | Versioned canonical contracts and generated types |
| Monolithic/local development | Docker Compose microservice environment |

# 2. Module Work Packages

Work should be assigned by module. Team members can redistribute modules without changing the architecture, provided module boundaries and shared contracts remain intact.

## A. Contracts

**Purpose:** Single source of truth for message schemas, topology IDs, enums, generated types and fixtures.

- Maintain envelope, telemetry, control-command, control-ack, graph-state, metrics, health and experiment schemas.
- Maintain four_intersection_v1 topology and canonical node/edge/approach/movement/phase IDs.
- Generate Python/Pydantic and TypeScript types from schemas.
- Provide contract validation tests and golden fixtures.

**Definition of done:** No service manually redefines shared telemetry, control, topology or metrics structures.

## B. MQTT / Infrastructure

**Purpose:** Mosquitto-based real-time event transport and Docker network configuration.

- Provide broker configuration, local development credentials/configuration and health checks.
- Standardize telemetry, control, acknowledgement, state, metrics, health and event topics.
- Provide integration tests that publish valid telemetry and verify consumer receipt.

**Definition of done:** Any compliant producer can publish telemetry and Graph-Control can consume it without implementation-specific knowledge.

## C. Simulator

**Purpose:** SUMO digital twin, controlled traffic scenarios and ground-truth benchmark source.

- Create the 4-intersection network and canonical mappings to topology IDs.
- Implement Balanced, Directional Surge and Downstream Congestion scenarios.
- Publish canonical telemetry from SUMO state.
- Consume ControlCommand and apply traffic-light phases.
- Export ground-truth traffic metrics.

**Definition of done:** SUMO runs as a closed-loop participant using only canonical EdgeFlow contracts.

## D. Graph-Control

**Purpose:** Global traffic graph state, freshness handling and interchangeable control policies.

- Maintain $G_t=(V,E,X_t)$ using queue, flow, speed and occupancy features.

- Implement FixedTimeController and MaxPressureController behind the same interface.
- Apply minimum/maximum green, yellow/all-red transition constraints and stale-telemetry rules.
- Publish ControlCommand, graph state and control metrics.
- Add RL/PressLight/MARL only after the baseline is validated.

**Definition of done:** Fixed-Time and Max-Pressure can be switched without changes to other services.

### E. Vision

**Purpose:** Camera/video perception adapter that emits the same traffic-state semantics as the simulator.

- Implement VideoSource abstraction: file, webcam, and later PiCamera.
- Implement detector, tracker, lane/ROI assignment and traffic-state extraction.
- Estimate queue, flow, speed, occupancy and confidence.
- Publish canonical telemetry; defer final Homography calibration until physical geometry is known.

**Definition of done:** Replacing SUMO telemetry with vision telemetry requires zero changes to Graph-Control.

### F. Actuator

**Purpose:** Safe signal execution independent of optimization logic.

- Implement SimulationActuator and VirtualActuator first; add GPIO/physical adapter later.
- Own signal transition state machine: current green -> yellow -> all-red -> new green.
- Validate commands, enforce safety timing, publish acknowledgements and implement fallback behavior.

**Definition of done:** The same ControlCommand can drive SUMO, a virtual signal, or future physical signal hardware.

### G. API / Persistence

**Purpose:** System control plane, experiment records, historical querying and dashboard bridge.

- Expose health/readiness, experiment, controller-mode, network-state and metrics endpoints.
- Use SQLite initially; only API owns persistent experiment storage.
- Bridge live state to the dashboard through WebSocket.

**Definition of done:** Experiments and historical results are accessible without direct access to internal service state.

### H. Dashboard

**Purpose:** IOC-style visualization for traffic state, system health and benchmark comparison.

- Render network topology and congestion state.
- Show phase, queue, flow, speed, pressure and telemetry age per intersection.
- Show service health and Fixed-Time vs EdgeFlow benchmark metrics.

**Definition of done:** A viewer can understand current traffic conditions, signal decisions and comparative performance from one screen.

### I. Experiment Runner

**Purpose:** Reproducible benchmark orchestration and artifact generation.

- Run experiment manifests specifying topology, scenario, seed, controller, duration and metrics.
- Execute repeated Fixed-Time vs Max-Pressure runs.
- Capture manifest, environment, metrics, events and summary artifacts.

**Definition of done:** The same manifest and seed reproduce the same scenario and comparable output.

### J. Testing / CI / Agent Guardrails

**Purpose:** Cross-module validation, Docker smoke tests and coding-agent safety rules.

- Maintain unit, contract, component, integration and end-to-end tests.
- Validate producer/consumer contracts in CI.
- Maintain AGENTS.md, architecture decision records and allowed-path task patterns for coding agents.
- Require docker compose smoke tests before merge.

**Definition of done:** The complete stack can be built, validated and smoke-tested automatically.

## 3. Shared Engineering Standards

### 3.1 Canonical topology and identifiers

- Use stable human-readable IDs such as J1, J1_N_IN, J1_N_TO_E, J1_PHASE_NS and E_J1_J2_EB.
- Never allow separate modules to invent unrelated numeric mappings without an explicit canonical mapping.
- Keep the topology in a versioned JSON file consumed by simulator, graph-control, vision configuration, dashboard and experiments.

### 3.2 Standard message envelope and telemetry

```
{
 "schema_version": "1.0",
 "message_id": "unique-id",
 "timestamp_ms": 1727005820123,
 "source": {"service": "simulator", "instance": "simulator-1"},
 "trace_id": "exp42-step193",
 "payload": {}
}
```

```
{
 "node_id": "J1",
 "source_type": "simulation",
 "simulation_time_s": 46.0,
 "approaches": [
  {
   "approach_id": "J1_W_IN",
   "queue_vehicles": 8,
   "flow_veh_per_s": 0.42,
   "mean_speed_mps": 1.8,
   "occupancy_ratio": 0.63,
   "confidence": 1.0
  }
 ],
 "current_phase_id": "J1_PHASE_NS"
}
```

Allowed source types initially: simulation, vision, physical. Source type may change; traffic-state semantics must not.

### 3.3 Control command and acknowledgement

```
{
 "command_id": "cmd-1023",
 "node_id": "J1",
 "target_phase_id": "J1_PHASE_EW",
 "min_green_s": 7.0,
```

```
 "max_green_s": 20.0,
 "controller": "max_pressure"
}
```

- Acknowledgement statuses: ACCEPTED, APPLIED, REJECTED, EXPIRED, FAILED.
- Optimization services issue abstract phase commands; actuator adapters own physical or simulated signal sequencing.

## 3.4 Units, clocks, freshness and errors

| Quantity | Canonical unit |
|---|---|
| Distance | metres (m) |
| Speed | m/s |
| Flow | vehicles/s |
| Density | vehicles/m |
| Queue | vehicles |
| Duration | seconds |
| Latency | milliseconds |
| Occupancy / confidence | ratio 0-1 |
| Timestamp | UTC Unix milliseconds |

- Maintain both timestamp_ms (distributed-system time) and simulation_time_s (traffic simulation time).
- Initial telemetry period: 1 s. Initial stale threshold: 3 s.
- Canonical errors: E_SCHEMA_INVALID, E_UNKNOWN_NODE, E_UNKNOWN_EDGE, E_UNKNOWN_PHASE, E_STALE_TELEMETRY, E_CONTROLLER_TIMEOUT, E_INVALID_COMMAND, E_BROKER_DISCONNECTED, E_SENSOR_UNAVAILABLE, E_SIMULATOR_UNAVAILABLE.

## 3.5 MQTT, REST and WebSocket responsibilities

| Mechanism | Use |
|---|---|
| MQTT | Telemetry, control commands, acknowledgements, state events, live metrics and health events |
| REST / FastAPI | Experiment creation/configuration, controller selection, historical queries, health/debug endpoints |
| WebSocket | Live dashboard updates |

```
edgeflow/v1/telemetry/{node_id}
edgeflow/v1/control/{node_id}/command
edgeflow/v1/control/{node_id}/ack
edgeflow/v1/network/state
edgeflow/v1/metrics/network
edgeflow/v1/health/{service}
edgeflow/v1/events
```

# 4. Development Environment, Repository and Agent Workflow

## 4.1 Docker Compose baseline

- Use Docker Compose, not Kubernetes, for the competition development stack.
- Canonical service names: mqtt, api, graph-control, simulator, vision, actuator, dashboard.
- Recommended profiles: simulation, vision, hardware, dev.
- Physical processes do not need to be containerized if they implement the same contracts.

```
docker compose --profile simulation up --build
```

## 4.2 Repository structure

```
EdgeFlow-AI/
|-- AGENTS.md
|-- ARCHITECTURE.md
|-- DEVELOPMENT.md
|-- CONTRIBUTING.md
|-- README.md
|-- docker-compose.yml
|-- .env.example
|-- Makefile
|-- contracts/
|-- infra/mosquitto/
|-- services/
|   |-- api/
|   |-- graph-control/
|   |-- simulator/
|   |-- vision/
|   |-- actuator/
|   `-- dashboard/
|-- experiment-runner/
|-- experiments/
|-- tests/
|-- docs/adr/
`-- artifacts/
```

## 4.3 Coding-agent guardrails

1. Never redefine canonical schemas inside a service; use generated contract types.
2. Do not directly import implementation code from another microservice.
3. Communicate across services only through documented MQTT/HTTP contracts.
4. Do not change canonical IDs, units, topics or schemas without updating contracts and tests.
5. Keep controllers behind one common controller interface.
6. Keep simulator, vision and future sensors behind common telemetry semantics.
7. Keep virtual and physical actuators behind the common command interface.
8. Limit agent tasks to explicit modules/paths and acceptance criteria; avoid unrelated cross-service edits.
9. Require contract tests and Docker integration smoke tests before merge.

# 5. Implementation Roadmap and Validation

## Phase 1 - Weeks 1-2: Software-first digital twin

| Period | Primary outcome | Acceptance gate |
|---|---|---|

| Period | Primary outcome | Acceptance gate |
|---|---|---|
| Week 1 | Contracts, Docker infrastructure, SUMO topology, graph state, Fixed-Time, Max-Pressure, virtual actuator, API logging | docker compose --profile simulation up starts a closed-loop system that runs at least 30 minutes |
| Week 2 | Laptop vision, ROI/state extraction, dashboard, experiment runner, fault handling and controlled benchmarks | SUMO and Vision can both feed Graph-Control without controller changes; benchmark artifacts are reproducible |

```
SUMO
 -> canonical telemetry
 -> MQTT
 -> Graph-Control
 -> Max-Pressure
 -> ControlCommand
 -> Actuator/SUMO
 -> metrics / dashboard
```

## Phase 2 - Hardware integration

- Replace file/webcam source with Pi Camera or available camera source.
- Perform final Homography and physical ROI calibration after physical geometry is known.
- Implement GPIO/relay/physical actuator adapter if the equipment is available.
- Keep MQTT contracts, graph-control, API, dashboard, experiment definitions and logging unchanged.
- If one camera cannot reliably cover four physical intersections, use two reliable physical intersections plus the four-intersection SUMO graph.

## Phase 3 - Evaluation and competition preparation

- Primary comparison: Fixed-Time vs EdgeFlow Max-Pressure.
- Traffic metrics: average waiting time, average queue, maximum queue, throughput and stopped vehicles.
- System metrics: controller latency; later add vision FPS, inference latency and queue-estimation accuracy.
- Recommended benchmark matrix: 3 scenarios x 2 controllers x 5 random seeds = 30 simulation runs.
- Introduce RL/PressLight/MARL only after the deterministic benchmark is stable and explainable.

## 5.1 Experiment manifest and artifacts

```
experiment_id: directional_surge_mp_seed42

topology: four_intersection_v1
scenario:
 name: directional_surge
 duration_s: 900
 random_seed: 42
controller:
 type: max_pressure
 decision_interval_s: 1.0
metrics:
 - average_waiting_time
 - average_queue
 - maximum_queue
 - throughput
 - stopped_vehicles
```

```
artifacts/<experiment-id>/
|-- manifest.yaml
|-- environment.json
|-- metrics.csv
|-- events.jsonl
`-- summary.json
```

# 6. Module Workload Summary and Critical Path

| Module | Core workload | Completion criterion |
|---|---|---|
| Contracts | Schemas, topology, generated types, fixtures | All modules validate against one definition |
| MQTT | Broker, topics, health, integration config | Stable producer/consumer messaging |
| Simulator | SUMO topology, scenarios, telemetry/control adapter | Closed-loop SUMO control |
| Graph-Control | Graph state, freshness, Fixed-Time, Max-Pressure | Controllers switch without other-service changes |
| Vision | Source abstraction, detector/tracker, ROI and traffic state | Publishes simulator-compatible telemetry |
| Actuator | State machine, ACK, failsafe, virtual/physical adapters | Same commands work across actuator types |
| API | REST, SQLite, experiment/history, WebSocket | Stable system control/query interface |
| Dashboard | Topology, state, health, benchmark views | Live behavior is understandable |
| Experiment Runner | Manifests, repeatable runs, artifacts | Reproducible benchmark suite |
| Testing/CI | Contract, integration, E2E, agent guardrails | Whole stack validates automatically |

**Critical path: Contracts -> MQTT + Simulator + Graph-Control -> closed-loop digital twin -> API/Dashboard/Experiment Runner -> Vision compatibility -> physical adapters -> benchmark and competition demo.**

## Source Basis and Superseded Assumptions

This plan is compiled from the team's VEDC 2026 registration and preliminary proposal, the previous EdgeFlow implementation plans, the equipment/BOM report, and the subsequent architecture changes agreed during planning. **The older ESP32 + IR + ESP-NOW implementation is retained only as historical context and is no longer on the critical path.**

- Registration / preliminary proposal: EdgeFlow AI as embedded-AI, graph-based adaptive traffic control.
- Previous implementation plan: SUMO/CityFlow, centralized FastAPI/MQTT variant, graph control, vision, dashboard, benchmarking and optional RL.
- Original equipment report: Raspberry Pi 5, camera, ESP32 nodes and IR sensors; the unavailable ESP32/IR dependency is removed from the revised implementation.
- Current team decision: software-first digital twin for Weeks 1-2, then hardware integration through stable adapters.